

Proposed methods

ECN (DCTCP):

Comparison to Homa's TCP implementation (what they did wrong):

The Homa paper argues that TCP's congestion control is inherently reactive—it waits for packet loss to slow down. Nonetheless, the paper still implements DCTCP as a part of their benchmarking, and we believe that this is a good place to start from. There is no 'benefit' from using CUBIC (or other algorithms) over ECN.

Why we believe it will improve performance:

DCTCP (Data Center TCP) uses ECN and allows the sender to scale back its rate smoothly, maintaining high throughput while keeping switch queues nearly empty. This mimics Homa's goal of low queueing delay.

Feasibility (how to setup in CloudLab / linux)

- On all nodes: `sudo modprobe tcp_dctcp`
- Set congestion control: `sudo sysctl -w net.ipv4.tcp_congestion_control=dctcp`
- Enable ECN: `sudo sysctl -w net.ipv4.tcp_ecn=1`

TCP Fast Open

Comparison to Homa's TCP implementation (what they did wrong):

Homa is praised for "Zero-RTT" starts. The Homa paper's TCP baseline usually incurs a ~1 RTT penalty before the first byte of data is even processed by the server application.

Why we believe it will improve performance:

TFO allows the initial SYN packet to contain the application payload. For the small, bursty messages you are testing, this removes the "handshake tax." It specifically targets the "short-flow" latency that Homa claims TCP cannot handle efficiently.

Feasibility (how to setup in CloudLab / linux)

1. Kernel: `sudo sysctl -w net.ipv4.tcp_fastopen=3`
2. Client Code: Use `sendto()` with the `MSG_FASTOPEN` flag.
3. Server Code: Set `setsockopt` with `TCP_FASTOPEN` before the `listen()` call.

Connection Pooling:

Comparison to Homa's TCP implementation (what they did wrong):

Homa's authors point out that TCP is "connection-oriented," implying a high overhead for every new RPC. In many benchmarks, if a new TCP connection is opened for every request, the overhead of the 3-way handshake and slow-start dominates the results.

Why we believe it will improve performance:

By "pooling" or reusing existing established sockets for multiple RPCs, you bypass the connection setup phase and, more importantly, stay in the Congestion Avoidance phase (large window) rather than restarting in Slow Start for every message. This levels the playing field against Homa's connectionless nature.

Feasibility (how to setup in CloudLab / linux)

This is an application-layer change (adjusting the trace itself).

Manually build the application such that it works for the particular trace (hadoop trace schema). Because we know the sender and receivers explicitly, we can implement tcp pooling efficiently

Extended Goals

Multiplexing (Xander):

Comparison to Homa's TCP implementation (what they did wrong):

The Homa paper critiques TCP for Head-of-Line (HoL) Blocking at the message level. In their benchmarks, if a large "background" RPC is being sent over a TCP connection, a small "latency-critical" RPC must wait for the entire previous message to finish before it can even begin. Homa solves this with packet-level interleaving. HTTP/2 brings this same philosophy to TCP by breaking messages into independent frames, allowing a small metadata request to "cut in" between frames of a large Hadoop block transfer on the same socket.

Why we believe it will improve performance:

Multiplexing allows the sender to maintain a single, high-concurrency pipe to each of the 9 receivers. While connection pooling reuses sockets, it still usually processes requests serially per socket. HTTP/2 allows the sender to fire off dozens of concurrent Hadoop tasks simultaneously over one DCTCP-managed flow. This keeps the DCTCP congestion window "warm" and provides a more stable ECN signal to the switch than a burst of many independent, competing TCP flows which often cause the "Incast" drops Homa's authors highlight.

Feasibility (how to setup in CloudLab / linux)

Most modern libraries support multiplexing via http2 (h2 API in python), which is ideal for CloudLab benchmarks.

Staggered Request Scheduling (Shabib):

Comparison to Homa's TCP implementation (what they did wrong):

Homa's authors identified TCP incast collapse as a failure in datacenter RPC workloads for TCP. When a server requests data from multiple other servers at the same time, all the responses are given back to the original server as fast as possible, which may end up overwhelming the original's buffer depending on the number of requests sent out. This buffer congestion causes dropped packets, latency spikes, and retransmission timeouts which all cause major latency compared to Homa.

Why we believe it will improve performance:

If we can control when each server is allowed to respond to the requesting server through a token-based scheduler in the application layer, the buffer of the requester will never be overwhelmed by a flood of responses. This change will eliminate retransmission cascades, causing the buffer to not be overwhelmed and decrease the latency spikes and dropped packets, reducing the amount of high percentile latency caused by such scenarios. This

improvement should directly allow TCP to improve at those percentiles, which is where Homa claimed it has the highest improvement.

Feasibility (how to setup in CloudLab / linux)

This is a purely application layer solution

1. Instead of simultaneous broadcasting the application will send requests in staggered batches using a tunable request number. For example requests = 3 will cause only 3 requests to be sent out simultaneously at any given time
2. Servers receiving requests can only send their responses after receiving an explicit request from the requesting server
3. The window of requests will be tunable from 1 to the full number of servers, allowing us to identify the exact amount of servers that can cause incast collapse on our network and create performance curves for analysis
4. This will be implemented on a separate tcp connection, dividing the scheduling logic from the actual data packets.

TCP Session load balancing (Krishna):

Comparison to Homa's TCP implementation (what they did wrong):

The authors used a single TCP session per receiver and cited head-of-line blocking as a fundamental TCP flaw. As the blog notes, this is a well-understood problem with a well-known solution: parallel sessions. The Homa paper's TCP baseline conflates a poor deployment choice with a fundamental protocol limitation.

Compromise between session load balancing and packet spraying:

True packet spraying distributes individual packets across multiple paths at the network layer, requiring kernel modifications or a connectionless protocol. Session load balancing is a user-space approximation: it distributes RPCs across sessions rather than packets across paths. For the short-message-dominated workloads we test, this distinction matters less since each RPC is small and the next one will likely land on a different session anyway.

Why we believe it will improve performance:

By maintaining a pool of N sessions per receiver and assigning each trace unit to the session with the least outstanding bytes, we avoid large RPCs blocking small ones on the same session. Since message sizes are known at trace-replay time, load tracking is straightforward in user space with no kernel involvement.

Feasibility (how to setup in CloudLab / Linux):

This is a purely application-layer change. The number of sessions per receiver is tunable from 1 to N, letting us generate performance curves as a function of session count. Each session maintains a byte counter that increments on assignment and decrements on response receipt, with new trace units routed to the lowest-load session. This integrates naturally with the connection pooling optimization already described.

Testing Methods and Platforms (Cloudlab):

Cloudlab Hardware:

We want to mimic the test setup established in the Homa paper in hopes of faithfulness in implementation and direct comparison in results. Given this we will be provisioning

- CloudLab xl170 nodes module in the Utah Cluster
- 10 nodes: 1 client + 9 servers
- Linux OS: Ubuntu 20.04 (to approximate Linux 5.4.80)

Data Configuration:

Our data: a synthetic trace generated from Hadoop-cluster size distribution (W4 in Homa paper)

Use a **synthetic trace generated from the Hadoop distribution** in the Homa paper (W4) rather than trying to reconstruct full Hadoop semantics from raw job logs. That keeps the experiment faithful to the Homa paper while still being easy to implement

Replay it as a **partition/aggregate fan-out workload**:

- the client represents an aggregator / reducer / coordinator
- each server returns one response whose size is sampled from the W4 distribution

Workload data

- Message sizes: sample from W4 (Facebook Hadoop) distribution
- Request size: fixed ~100 B
- Response size: sampled from W4 (truncate at 1 MB)
- 9 client nodes all issuing requests to a single server node.

Arrival pattern

- Generate RPC arrivals via Poisson process
- Tune λ to control offered load (e.g., ~30%, 60%, 80–90% NIC utilization)

Traffic structure

- Each RPC:
 client → server request → server → client response
- Maintain 32–128 outstanding RPCs to keep network saturated.

Experiment duration

- 5 seconds per run (same style as Homa paper)
- 5 runs per configuration

Total workload size

- Expect roughly 50k–500k RPCs per 5-second run depending on λ and message sizes.

Expected Initial Behaviors:

Why this will cause HoL blocking

Because W4 is exactly the kind of workload that hurts TCP-style transports:

- it has a mixed-size distribution with lots of short messages and a nontrivial tail of large ones up to 1 MB
- 9 client nodes all issuing requests to a single server node.
- at the application level, the client is doing a fan-in barrier: the query completes only after the slowest shard returns, so one delayed large response stalls the whole query.

HoL may manifest in two ways:

- **Transport/socket HoL**
A large response on a TCP connection occupies the receive path and delays later useful bytes behind it.
- **RPC/query-level HoL**
The client cannot finish the fan-out request until the last server reply arrives, so one large or loss-hit flow becomes the tail.

Why we expect to see loss:

The Homa paper explicitly reports packet drops under stressful workloads and discusses sensitivity to buffer conditions; it also uses short repeated benchmark runs rather than long one-off experiments. Furthermore, reasons we expect to still lose packets is for the following three reasons:

1. DCTCP reacts after a burst has already started

- DCTCP reduces its window based on ECN marks returned in ACKs. That means the first incast burst can already fill the queue before the senders back off.

2. Bare-bones Linux senders tend to burst

- If you initially use ordinary sockets with no careful pacing strategy, the senders will emit bursts shaped by the kernel/NIC transmit path rather than smooth grant-based scheduling. Under synchronized 9-way fan-in, those bursts pile up at the receiver-facing queue.

3. W4 has enough large responses to overflow shallow queues during synchronized fan-in

- The DCTCP paper notes that short and medium flows suffer when long flows consume switch buffer space, and that query traffic experiences incast impairment. W4's long tail means some epochs will contain several much-larger responses mixed with small ones, which is exactly how you get queue spikes, CE marks, and—before adaptation catches up—drops.

Hello Dr. Saeed,

We hope you are doing well. We wanted to briefly follow up on our meeting last Thursday and share our updated project scope and experimental plan (for more specifics, please reference our *revised* outline document:

https://docs.google.com/document/d/1HLm8vTSP_AKXFKQRCMKOxojpTFG7M0RIn0Ls3ajcEWs/edit?tab=t.0).

Minimum deliverables discussed:

- DCTCP
- TCP Fast Open
- TCP connection pooling

Additional optimizations:

We plan to divide the work by exploring individually, and assist others if finished early.

- **Xander:** HTTP/2 multiplexing to reduce message-level HoL blocking by allowing multiple RPCs to interleave on a single TCP flow.
 - Phase 1: HTTP/2 multiplexing over a single persistent TCP connection (no connection pooling).
 - Phase 2: Combine HTTP/2 multiplexing with TCP connection pooling.
 - Phase 3: Tune the number of HTTP/2 sessions per receiver and the number of concurrent streams per session
- **Shabib:** Staggered request scheduling to mitigate incast collapse by controlling response timing with a token-based application-layer scheduler.
 - Phase 1: static staggered scheduling (fixed offset delays)
 - Phase 2: token-based scheduler
 - Phase 3: adaptive token control (noting packet loss + ECN marks)
- **Krishna:** TCP session load balancing by distributing RPCs across multiple sessions per receiver to prevent large transfers from blocking smaller ones.
 - Phase 1: Round-robin across static pooling
 - Phase 2: Load-aware session scheduling
 - Phase 3: Size-aware scheduling

Testbed and methodology:

We will reproduce the evaluation environment used in the Homa paper using CloudLab (10 nodes: 1 client, 9 servers) and replay a synthetic workload based on the Hadoop W4 message-size distribution. The workload will follow a partition/aggregate pattern where the client fans out requests to all servers and waits for all responses, allowing us to reproduce head-of-line blocking, incast behavior, and queue pressure similar to the original evaluation.

The attached document describes the full experimental setup, workload generation, and reasoning behind the expected HoL blocking, incast, and packet loss behavior.

Please let us know if this scope and methodology align with your expectations, and if our proposed workload seems reasonable or overly ambitious.

Best,

Krishna, Shabib, and Xander